MODULE 04 · DOMAIN 1 - SDLC AUTOMATION

Testing in the Pipeline

Which tests run at which stage, how a test actually stops a release, and how to test security, load, resilience and production itself.

DOMAIN 1 of 6 EXAM WEIGHT 22% MODULE 4 of 34

admin@sthithapragna.com

A test only protects a pipeline if its failure **stops the pipeline**. Everything else is a report nobody reads.

Which tests run where

STAGE	TESTS	WHY HERE
Pull request	Lint, unit, SAST, secrets scan	Seconds to run; fix before merge
Build	Unit, coverage, dependency and image scans	Every artifact checked once
Staging	Integration, API, UI, load, dynamic scans	Needs a running, realistic system
Production	Smoke tests, synthetic canaries, alarms	Proves the real thing is healthy

ON THE EXAM

Cheap and fast tests run first. A slow test placed before a fast one wastes every failing run's time.

How a failing test stops a release

01
Test runner
One test fails. The runner exits non-zero.

›

02
Command
CodeBuild sees the non-zero exit code.

›

03
Phase and build
The phase fails, so the build fails.

›

04
Action and stage
The pipeline action fails, then the stage.

›

05
Release halted
Later stages never start.

ON THE EXAM

Every link is an exit code. Break any one of them and a failing test becomes invisible to the pipeline.

Three ways to swallow an exit code

```
1  # 1. Explicitly ignored
2  - npm test || true
3
4  # 2. Lost in a pipe: tee succeeds, so the line succeeds
5  - pytest | tee results.txt
6
7  # 3. Run in the background, never waited for
8  - ./integration-tests.sh &
9
10 # Fix for 2: run under bash (env: shell: bash), then fail the pipe on any failure
11 - set -o pipefail && pytest | tee results.txt
```

5 A pipe's exit status is the last command's. tee always succeeds.

11 pipefail is a bash option: the pipe returns the first failing command's status.

ON THE EXAM

'Tests fail but the build passes' is a diagnosis question. Look for a pipe, an || true, or a background job.

Where a test can sit in CodePipeline

MECHANISM	BEHAVIOUR ON FAILURE	GOOD FOR
CodeBuild test action	Fails the stage	Unit, integration, load tests
Parallel test actions	Same run order, any failure fails the stage	Independent suites at once
Device Farm action	Fails the stage	Mobile apps on real devices
CodeBuild or Commands rule	Condition fails: roll back or block entry	Tests after a deploy
Lambda or Step Functions invoke	Reports job failure	Custom checks

ON THE EXAM

Independent suites taking too long in sequence: give them the same run order so they run in parallel.

Integration tests need somewhere to run

- Create a **short-lived test environment** from the same IaC templates, test it, tear it down.
- Tests against private databases need the CodeBuild project in the **VPC**, as in module 3.
- Test credentials come from Secrets Manager. Test data should be seeded, not copied from production.
- A shared, long-lived test environment drifts, and tests start failing for reasons unrelated to the change.

> ON THE EXAM
>
> Environment per pull request, from IaC, destroyed afterwards: consistent, isolated, and you only pay while testing.

Testing user interfaces and mobile apps

- **AWS Device Farm** runs automated tests on real phones and tablets, many models at once.
- It plugs into CodePipeline as a test action, and scales with the number of devices you choose.
- Browser tests can run headless inside CodeBuild with Selenium or Playwright.
- A fleet of EC2 instances running emulators is the high-overhead distractor.

> ON THE EXAM
>
> Many device types, scaling test load, least management: Device Farm. Emulators on EC2 or ECS lose on overhead.

Security testing, by type

TYPE	WHAT IT FINDS	AWS OPTION
SAST	Flaws in your own source code	Inspector code security, Q Developer review
SCA	Vulnerable third-party dependencies	Inspector code security and image scanning
IaC scanning	Misconfigured templates	Inspector, cfn-guard or cfn-lint in the build
Image scanning	Vulnerable OS and language packages	ECR basic or enhanced scanning
DAST	Flaws in the running application	Third-party tools against staging

ON THE EXAM

Static scans need only code, so they run early. Dynamic scans need a running system, so they run in staging.

The catalogue has moved: code review

OLDER MATERIAL SAYS	WHAT HAPPENED	USE INSTEAD
CodeGuru Reviewer	Closed to new customers, November 2025	Amazon Q Developer code review
CodeGuru Security	Discontinued, 20 November 2025	Inspector code security
Inspector: EC2, ECR, Lambda only	Code security added, June 2025	SAST, SCA, IaC via GitHub, GitLab

THE RULE

If an exam option names CodeGuru for automated code review, it reflects the guide. Choose it on the guide's terms.

ECR basic or enhanced scanning?

ENHANCED

Through Amazon Inspector

- OS and programming language packages
- Scan on push, and continuous rescans
- An EventBridge event per finding
- Findings flow to Security Hub. Paid

BASIC

Built into Amazon ECR

- Operating system packages only
- Scan on push, or manual
- One summary event per scan
- Free. Clair engine now deprecated

THE TIEBREAKER

'Newly published vulnerabilities in images already deployed' needs continuous rescanning, which only enhanced offers.

Turning a scan into a gate

- Scan on push is **asynchronous**. A pipeline that pushes then deploys can outrun the scan.
- Gate with a build step that **waits for the scan**, reads the findings, and exits non-zero on critical ones.
- Or scan inside the build, before the image is ever pushed.
- For images already deployed, an **EventBridge rule** on new critical findings notifies or remediates.

> ON THE EXAM
>
> A scan that only reports is a detective control. A scan whose result sets an exit code is a gate.

Load and performance testing

- Run in **staging**, sized like production, never against production during business hours.
- Scale staging **up before** the test and **back down after**, as pipeline steps. Pay only while testing.
- Generate load from CodeBuild, or the **Distributed Load Testing on AWS** solution for large scale.
- Fail the stage on a threshold: latency, error rate, or throughput below a target.

> ON THE EXAM
>
> Load test fails because staging scales too slowly: raise its minimum capacity before the test, restore it after.

Resilience testing

- **AWS Fault Injection Service** runs experiments: stop instances, add latency, throttle APIs.
- **Stop conditions** are CloudWatch alarms that halt an experiment if harm goes too far.
- Run experiments in staging from the pipeline, and check the system recovers without people.
- **Resilience Hub** assesses an application against RTO and RPO targets, and can run in the pipeline.

> ON THE EXAM
>
> 'Verify the application tolerates the loss of an Availability Zone before release': an FIS experiment in staging.

Testing in production, safely

- **CloudWatch Synthetics canaries** run scripted journeys or API calls on a schedule, from outside.
- A canary's failures feed an **alarm**, and the alarm can drive a rollback.
- Blue/green deployments can test the new version on a **test listener** before any real traffic.
- A canary deployment and a Synthetics canary are different things with the same name.

> ON THE EXAM
>
> 'Continuously verify a user journey works, with the least development effort': a Synthetics canary and an alarm.

A pipeline's CodeBuild test action runs this command so that results are both shown in the log and saved to a file: pytest | tee test-results.txt.

Last week several tests failed, yet the test action succeeded and the release was deployed to production. The buildspec has no on-failure settings.

What is the most likely cause, and fix?

A. The build timeout is too short; increase it so pytest can finish

B. Tests should run in post_build; move the command there

C. The pipeline needs a manual approval after the test action

D. The pipe returns tee's exit status; enable pipefail so pytest's failure propagates

◆ PAUSE AND DECIDE

What is the most likely cause, and fix?

A Longer timeout

A timeout would fail the build, not pass it. And the tests did run: they failed.

B Move to post_build

Same command, same pipe, same swallowed exit code. The phase is not the problem.

C Manual approval

Adds a human to catch what an exit code should have caught. Treats the symptom.

D pipefail

A pipe reports its last command's status, and tee succeeded. pipefail reports pytest's failure.

THE SIGNAL

Tests failed, build passed: something swallowed the exit code. A pipe is the usual culprit.

A company requires that no container image with critical vulnerabilities is ever deployed. ECR enhanced scanning with scan on push is enabled.

The pipeline builds and pushes the image, then deploys it to ECS in the next stage. Twice, an image was deployed before its scan finished and was later found to contain a critical vulnerability.

Which solution meets the requirement with the LEAST operational overhead?

A. Add a step after the push that waits for the image scan to complete and fails when critical findings exist

B. Switch the registry to basic scanning so that scans finish faster

C. Add a manual approval so the security team reviews findings before each deployment

D. Enable continuous scanning on the repository

◆ PAUSE AND DECIDE

Which solution meets the requirement with the LEAST operational overhead?

A Wait for the scan, fail on critical

Turns an asynchronous report into a synchronous gate. The deploy cannot start until the verdict exists.

B Basic scanning

Still asynchronous, and it misses language package vulnerabilities. Weaker and no gate.

C Manual approval

Would work, but adds a person to every release. The opposite of least overhead.

D Continuous scanning

Finds new vulnerabilities later. It does not stop the first deployment racing the first scan.

THE SIGNAL

Deployed before the scan finished: the scan is asynchronous. Make the pipeline wait for its verdict.

> An online retailer deploys to production several times a day. Twice last month, the checkout journey broke after a deployment, and customers reported it before the team noticed.
>
> The team wants the checkout journey exercised every five minutes, from outside the application, with automatic alerts when it fails. It wants the least development effort.

What should the team implement?

A. A Lambda function on an EventBridge schedule that runs a Selenium script and publishes a custom metric

B. A Device Farm test run triggered every five minutes

C. X-Ray tracing on the checkout service, with sampling set to 100%

D. A CloudWatch Synthetics canary scripting the checkout journey, with a CloudWatch alarm on its failures

◆ PAUSE AND DECIDE

What should the team implement?

A Scheduled Lambda with Selenium

Rebuilds a canary by hand: packaging a browser, scheduling, metrics. Real development effort.

B Device Farm every five minutes

Built for pre-release testing on devices, not for continuous production monitoring.

C X-Ray at 100%

Only sees requests customers make. It never exercises the journey itself.

D Synthetics canary with an alarm

Exactly its purpose: a scripted journey, on a schedule, from outside, with metrics for alarms.

THE SIGNAL

Scripted journey, on a schedule, from outside, with alerting: that sentence describes a Synthetics canary.

A pipeline load-tests a staging ECS service before production. Staging normally runs 2 tasks with target tracking scaling. Production runs up to 20 tasks at peak.

The load test keeps failing its latency threshold because staging scales out too slowly for the sudden test load. The team wants a realistic test at the lowest cost.

What should the team do?

A Run the staging service at 20 tasks permanently

B Add pipeline steps that raise staging's minimum task count to 20 before the test and restore it afterwards

C Run the load test against production during off-peak hours

D Relax the latency threshold until the test passes

◆ PAUSE AND DECIDE

What should the team do?

A 20 tasks permanently

Realistic, but pays for production-sized staging around the clock. Not lowest cost.

B Scale up before, restore after

Production-sized only while testing, then back to 2. Realistic and cheap.

C Test production off-peak

Risks real customers and real data. Never the intended answer for load testing.

D Relax the threshold

Makes the gate pass by removing it. The test no longer protects anything.

THE SIGNAL

Realistic capacity, but only while testing: change the capacity as pipeline steps on either side of the test.

Module 4 in five lines

- Cheap tests first. Static scans early; integration, load and dynamic scans in staging.
- Exit codes are the gate. Pipes, || true and background jobs break the chain.
- Short-lived test environments from IaC. Device Farm for many devices.
- CodeGuru is gone: Inspector code security and Q Developer review replace it.
- Asynchronous scans need a waiting step to become gates. Synthetics canaries watch production.

> ON THE EXAM
>
> Next, module 5: artifacts and images. S3, CodeArtifact, ECR, and building golden images with Image Builder.